

RealEstateAI

Estimateur de prix immobilier — Île-de-France

Basé sur les données DVF open data

Prototype v1.0 · Mars 2026

github.com/khadija10/RealStateAI

DOSSIER DE PROTOTYPAGE — RealEstateAI

Prototype v1.0 — Mars 2026

Dépôt Git : <https://github.com/khadija10/RealStateAI>

1. CONTEXTE

Présentation du contexte

Le marché immobilier francilien est complexe, fragmenté et opaque pour les particuliers. Les transactions immobilières en Île-de-France représentent plusieurs centaines de milliers d'opérations annuelles, mais il n'existe pas d'outil simple, gratuit et basé sur des données réelles permettant à un vendeur ou acheteur d'obtenir rapidement une estimation de prix fiable.

L'État français publie en open data les **Demandes de Valeurs Foncières (DVF)**, un registre exhaustif de toutes les ventes immobilières notariées. Ce jeu de données constitue une base de travail exceptionnelle, encore peu exploitée sous forme d'application accessible au grand public.

Objectifs du POC

Valider la faisabilité technique d'un estimateur de prix immobilier basé sur les données DVF réelles, accessible via une interface web, et retournant en quelques secondes une fourchette de prix de marché pour un bien donné en Île-de-France.

2. LE POC

Hypothèses du POC

- Les données DVF 2023–2024 contiennent suffisamment de transactions (≥ 10) par commune et type de bien pour produire une estimation statistiquement significative
- Le prix d'un bien est linéairement proportionnel à sa surface (hypothèse simplificatrice acceptée pour ce stade)
- La commune est le niveau de granularité géographique suffisant pour le MVP
- Les données DVF, une fois nettoyées, sont représentatives du marché réel sans biais majeur
- L'infrastructure Docker garantit une reproductibilité totale du déploiement local

Ce que le POC cherche à valider

- Faisabilité du pipeline complet : données brutes DVF → nettoyage → API → interface utilisateur
- Pertinence d'une estimation statistique (médiane des prix au m²) comme approximation du prix de marché
- Comparaison baseline statistique vs modèle Machine Learning simple (RandomForest)
- Ergonomie et utilisabilité d'une interface de saisie / résultat

Éléments hors périmètre

- Géolocalisation fine (rue, quartier, étage, exposition)
- Historique et persistance des estimations (base de données)
- Données socio-économiques complémentaires (INSEE, revenus, etc.)
- Déploiement cloud / production
- Authentification utilisateur
- Couverture géographique hors Île-de-France

Contraintes

- Données partielles : uniquement 2 années (2023–2024), uniquement maisons et appartements
- Pas de filtre outliers automatique sur les prix extrêmes dans la version actuelle
- Modèle ML limité par le faible nombre de features (commune, type, surface uniquement)

3. LES DONNÉES

Origine des données

Source : Demandes de Valeurs Foncières (DVF) — données ouvertes publiées par la Direction Générale des Finances Publiques (DGFIP) sur data.gouv.fr. Fichiers annuels au format TXT délimité par `	, couvrant l'intégralité des mutations immobilières notariées en France. Millésimes disponibles dans le projet : 2022, 2023, 2024, 2025-S1. Dataset retenu : DVF 2023–2024.
---	---

Description rapide des données (après nettoyage)

Indicateur	Valeur
Transactions finales (2023–2024, IDF)	257 354
Types de biens	Appartement, Maison
Départements couverts	75, 77, 78, 91, 92, 93, 94, 95
Variables clés	commune, type_bien, surface_m2, prix_vente, prix_au_m2
Doublons (adresse + date + prix)	0
Lignes avec prix ≤ 0 après nettoyage	0
Présence d'outliers extrêmes	Oui (jusqu'à 2.5 × 10 ⁸ €/m ²)

Traitements effectués

1. Lecture des fichiers TXT (séparateur `), sélection des colonnes utiles uniquement

1. Suppression des lignes sans Valeur foncière ou Surface réelle bâtie
2. Conversion des types (virgule → point pour les numériques, parsing des dates)
3. Filtrage géographique : codes postaux à 5 chiffres, départements IDF uniquement
4. Filtrage métier : type_bien ∈ {Appartement → apartment, Maison → house}
5. Calcul de prix_au_m2 = Valeur foncière / Surface (arrondi à 2 décimales)
6. Construction de la colonne adresse (concaténation normalisée)
7. Suppression des doublons stricts (adresse + date + prix)
8. Export CSV : DVF_clean_2023_2024.csv

4. TECHNIQUE

Architecture de la solution

```
UTILISATEUR (Browser) – http://localhost:8501
  |
  ▼ HTTP
FRONTEND – React + Vite + TypeScript (port 8501)
  ├── Home.tsx          (page d'accueil)
  ├── Estimator.tsx     (formulaire d'estimation)
  └── EstimationResult.tsx (affichage résultat)
    |
    ▼ JSON REST
BACKEND – FastAPI (port 8000)
  ├── GET /api/health
  ├── GET /api/metadata/communes
  └── POST /api/predictions/estimate
      |
      ├──> Modèle ML (RandomForest .pkl)
      └──> DVF CSV (257 354 lignes)

Infrastructure : Docker + Docker Compose
```

Flux d'estimation (ordre de priorité)

Requête POST /api/predictions/estimate



Modèle ML disponible ?

OUI → Prédiction RandomForest Pipeline

NON → DVF chargé ?

OUI → Recherche transactions similaires
→ médiane prix/m² × surface

NON → Fallback mock (7 000 €/m²)

Algorithme de recherche DVF

1. Filtre strict : commune + type_bien + surface ±20%
2. Si vide → élargissement ±35%, puis ±50%
3. Si vide → fallback sur préfixe commune
4. Calcul :
 - prix_estimé = médiane(prix_au_m2) × surface
 - borne_basse = Q1(prix_au_m2) × surface
 - borne_haute = Q3(prix_au_m2) × surface
 - confiance = min(1.0 ; n / 50)

Outils et langages utilisés

Composant	Technologie	Version
Langage backend	Python	3.12
API REST	FastAPI	0.128
Validation données	Pydantic	2.12
Traitement données	Pandas	2.3
Machine Learning	scikit-learn	1.5.2
Sérialisation modèle	joblib	1.4
Frontend	React + Vite + TypeScript	—
CSS	Tailwind CSS	—
Conteneurisation	Docker + Docker Compose	—
Tests	pytest	9.0
Serveur ASGI	Uvicorn	0.40

Modèle Machine Learning

- Algorithme : `RandomForestRegressor` (200 arbres, `random_state=42`, `n_jobs=-1`)
- Features : `commune` (OneHotEncoded) + `type_bien` (OneHotEncoded) + `surface_m2`
- Cible : `prix_vente`
- Split : 80 % entraînement / 20 % validation

Résultats d'évaluation (50 000 lignes DVF 2023–2024)

Modèle	MAE (€)	MAPE (%)
Baseline — médiane prix/m² par commune + type	~190 882	~158.5 %
RandomForest — commune + type + surface	~244 177	~182.0 %

La baseline statistique surpasse le RandomForest à ce stade. Les MAPE élevés s'expliquent par les outliers non filtrés et le manque de features discriminantes.

Justification des choix

FastAPI a été retenu pour sa légèreté, sa documentation automatique (Swagger) et ses performances asynchrones. React + Vite offre un développement frontend réactif, Tailwind CSS permettant un prototypage UI rapide. Le CSV remplace une base de données pour la simplicité de déploiement en phase prototype. scikit-learn fournit des pipelines reproductibles avec préprocessing intégré. Docker garantit une reproductibilité totale de l'environnement.

Alternative non retenue : Streamlit (frontend) — initialement envisagé pour sa simplicité Python, remplacé par React pour plus de contrôle UI et de maintenabilité.

5. MANUEL D'UTILISATION

Lien Git : <https://github.com/khadija10/RealStateAI> — Branche `main` — Tag `v1.0`

Lien prototype : <http://localhost:8501> (après `docker compose up --build`)

API Swagger : <http://localhost:8000/docs>

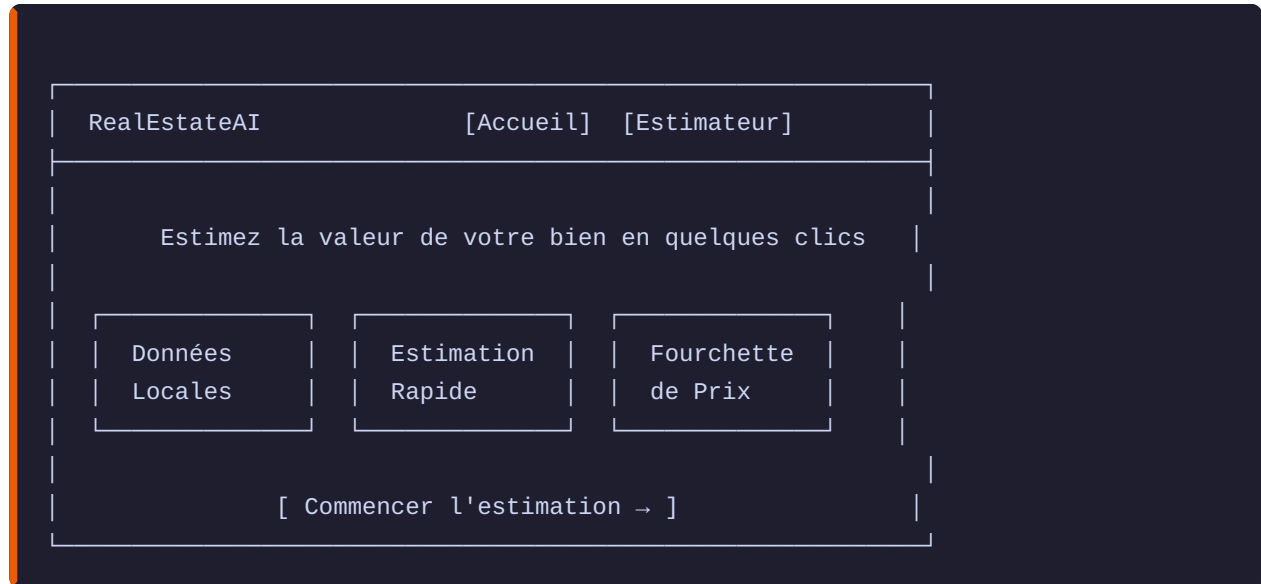
Lancement rapide

```
git clone https://github.com/khadija10/RealStateAI.git
cd RealStateAI
python3 -m venv data_env && source data_env/bin/activate
pip install -r requirements.txt
# Générer le CSV nettoyé
cd "Backend/Data pipeline/Traitement"
python3 -c "from dvf_data_pipeline import pipeline_dvf; pipeline_dvf('../Data/', 2024)"
cd ~/RealStateAI
# Lancer l'application
sudo docker compose up --build -d
```

Fonctionnalités

Fonctionnalité 1 — Page d'accueil

Présente les 3 piliers de l'outil (Données locales, Estimation rapide, Fourchette de prix) et oriente l'utilisateur vers le formulaire.



Fonctionnalité 2 — Formulaire d'estimation

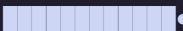
L'utilisateur renseigne 5 champs pour décrire son bien :

Champ	Type	Exemple
Commune	Dropdown (depuis DVF)	PARIS 10
Adresse	Texte libre (optionnel)	21 rue Proudhon, 93210 Saint-Denis
Surface (m²)	Numérique (±5 par bouton)	65
Nombre de pièces	Numérique (±1 par bouton)	3
Type de bien	Dropdown	Appartement / Maison / Studio

LOCALISATION		Votre estimation apparaîtra ici [🏠] Renseignez le formulaire puis lancez l'estimation.
Commune :	[PARIS 10 ▼]	
Adresse :	[.....]	
CARACTÉRISTIQUES		
Surface :	[-] [65] [+] m²	
Pièces :	[-] [3] [+]	
Type :	[Appartement ▼]	
[✨ Lancer l'estimation]		

Fonctionnalité 3 — Résultat d'estimation

Après soumission, le panneau de droite affiche le résultat structuré :

Estimation — PARIS 10	
€ 458 250	
App. · 65 m² · 3 pièces	
€ 7 050	3
PRIX / M²	PIÈCES
65 m²	App.
SURFACE	TYPE
Fourchette :	
Min — [] — Max	
€ 389 012	€ 526 987
↑ € 458 250	
Modèle : dvf	Confiance : 85%

Informations affichées : prix estimé total · prix au m² · surface · pièces · type · barre fourchette Q1–Q3 · modèle utilisé (**dvf** / **ml-simple** / **mock**) · score de confiance.

Fonctionnalité 4 — API REST (accès développeur)

Endpoint	Méthode	Rôle
<code>/api/health</code>	GET	Statut backend, DVF chargé, modèle ML
<code>/api/metadata/communes</code>	GET	Liste des communes disponibles
<code>/api/predictions/estimate</code>	POST	Estimation de prix (JSON)

Exemple de requête / réponse :

```
// POST /api/predictions/estimate
{
  "area_m2": 65,
  "rooms": 3,
  "property_type": "apartment",
  "commune": "PARIS 10"
}

// Réponse
{
  "predicted_price": 458250.00,
  "price_per_m2": 7050.00,
  "confidence_interval": {
    "lower": 389012.00,
    "upper": 526987.00,
    "confidence": 85
  },
  "model": "dvf"
}
```

Avancement global du prototype

Module	Avancement
Architecture	80 %
Backend API	75 %
Frontend	70 %
Pipeline Data	50 %
Modèle ML réel	20 %
Production ready	30 %
Global	≈ 60 %

Limites identifiées

Limite	Impact	Priorité
Pas de filtre outliers automatique (IQR)	Estimation biaisée sur petits échantillons	Haute
ML RandomForest moins précis que la baseline	Modèle non exploité en production	Haute
Granularité commune uniquement	Imprécis pour Paris intra-muros	Haute
Pas de base de données (CSV)	Pas de persistance, scalabilité limitée	Moyenne
2 lignes avec $\text{prix_au_m2} \leq 0$ résiduelles	Bruit marginal	Basse
Pas de déploiement cloud	Usage local uniquement	Moyenne

Pistes d'amélioration

1. **Filtrage outliers** : appliquer un filtre IQR ($Q1 - 1.5 \times IQR$; $Q3 + 1.5 \times IQR$) sur `prix_au_m2`

2. **Enrichissement features** : année de transaction, département, données INSEE (revenus, chômage)
 3. **Modèle plus performant** : XGBoost / LightGBM avec validation croisée — objectif $MAPE < 30 \%$
 4. **Granularité géographique** : passer au niveau code postal (arrondissement Paris) voire IRIS
 5. **Base de données** : intégrer PostgreSQL (schéma déjà présent dans `db/init.sql`)
 6. **Carte interactive** : visualisation des transactions similaires sur carte IGN/Leaflet
 7. **Déploiement cloud** : CI/CD GitHub Actions → VPS ou cloud (Render, Railway, OVH)
 8. **Tests automatisés** : couverture unitaire complète des utils et endpoints
-

Dossier rédigé le 12 mars 2026 — Prototype v1.0

Dépôt : <https://github.com/khadija10/RealStateAI>